NoSQL and MongoDB

Structured vs. Semi-Structured vs. Unstructured Data

- **Structured Data**

Data that is **organized in a fixed format** (rows & columns).
It follows a **strict schema**.
Characteristics:
Stored in tables
Easy to search with SQL
Clearly defined fields
Examples:
Student database table
Bank transactions
Employee records
Used In:
Relational databases (MySQL, PostgreSQL)
Banking systems

- **Semi-Structured Data**

Data that does **not follow a strict table structure**, but still has some organization (tags, keys).

Flexible schema.

Characteristics:

Self-describing

Uses tags or key-value pairs

Not stored in traditional tables

Examples:

JSON

XML

NoSQL documents

Used In:

NoSQL databases (MongoDB)

APIs

Web applications

```json
{
  "name": "Ahmed",
  "age": 20,
  "courses": ["DB", "AI"]
}
```

Notice: Another student may have different fields.

- **Unstructured Data**

Data with **no predefined format or structure**.
Cannot be stored easily in tables.

Characteristics:

Large volume
Difficult to search directly
Needs special tools (AI, ML)

Examples:

Images
Videos
Audio files
Emails
Social media posts
PDF documents

Used In:

Social media platforms
Multimedia systems
Big data analytics

- NoSQL databases are built to manage large amounts of unstructured and semi-structured data with flexible schemas and high scalability.
- Do not rely on fixed tables or predefined schemas.
- Support flexible data models for diverse data types.
- Enable horizontal scaling for high performance.
- Well-suited for modern, data-intensive applications.

- SQL : Structured Query Language for Relational DB (RDBM)
- NoSQL: unstructured and semi-structured

- **Features of NoSQL Databases**
- **Dynamic schema:** Allows flexible data structures without requiring schema changes or migrations.
- **Performance:** Perfect for big data and real-time analytics and high volume applications.
- **Flexibility:** Supports dynamic schemas and multiple data types for adaptable data storage and retrieval.
- **Distributed and high availability:** Maintains data availability using automatic replication and fault tolerance across cluster nodes.
- **Column-based:** Data are organized into columns instead of rows (e.g., CASSANDRA).
- **Key-value-based:** Stores data as key–value pairs for simple and fast access (e.g., Redis).
- **Document-based:** Data are presented in flexible, semi-structured formats like JSON/BSON (e.g., MongoDB).
- **Horizontal scalability:** Supports larger datasets and higher traffic by adding nodes and distributing load across servers.

- NoSQL:

Next generation databases mostly addressing some of the points:

Non-relational

Distributed

Schema free

Horizontally scalable

- Vertical scalability: means adding more data means scale up vertically by adding more rows “ in RDBM”.
- Horizontal scalability:

means adding data means scale up horizontally by adding more columns “in NOSQL”

Supports larger datasets and higher traffic by adding nodes and distributing load across servers.

- Of course you can add columns in RDBM, but this needs much effort for changing the structure of the whole db which supposed to be stable.

SQL vs. NoSQL

1. Data Model

- **SQL**
- Uses **tables** (rows & columns)
- Fixed **schema**
- **Example (Table):**

```
Students
--------------------------------
ID | Name    | Age | Department
1  | Ahmed   | 20  | CS
2  | Sara    | 21  | IT
```

- **NoSQL**
- Uses **flexible data models** (document, key-value, column, graph)
- **Schema-less**
- **Example (Document – MongoDB):**

```json
{
  "id": 1,
  "name": "Ahmed",
  "age": 20,
  "department": "CS",
  "courses": ["DB", "AI"]
}
```

Note: If in the next object you need to add more attributes such as birth-date, you can add it simply without any need to change in db structure “schema-less”.

2. **Schema**

- **SQL:** Schema must be defined **before** inserting data

```sql
CREATE TABLE Students (
  ID INT,
  Name VARCHAR(50),
  Age INT
);
```

- **NoSQL:** No predefined schema, Each document can be different.

```json
{ "name": "Sara", "email": "sara@gmail.com" }
{ "name": "Ali", "phone": "010..." }
```

3. Consistency & Transactions

- **SQL**

Strong **ACID** properties

Always consistent

- **Example:**
  Bank transfer (money must not be lost)
- **NoSQL**

Uses **BASE** (eventual consistency)

- **Example:**
  Social media likes may update slightly later

4. Relationships

- **SQL**

Supports **JOINs**

```sql
SELECT s.name, c.course
FROM Students s
JOIN Courses c ON s.id = c.student_id;
```

- **NoSQL**

No JOINs (data is often embedded)

```json
{
  "name": "Ahmed",
  "courses": ["DB", "AI"]
}
```

Feature	SQL Project Example	NoSQL Project Example
Data type	Structured	Semi-structured / unstructured
Scaling	Vertical	Horizontal
Consistency	ACID	Eventual (BASE)
Users	Thousands	Millions
Examples	Banking, ERP, E-commerce orders	Social media, Netflix analytics, IoT, Amazon catalog

Think????

Use Case	SQL	NoSQL
Banking systems		
E-commerce orders		
Social media		
Big data / real-time		
IoT data		

Use Case	SQL	NoSQL
Banking systems	✅	❌
E-commerce orders	✅	❌
Social media	❌	✅
Big data / real-time	❌	✅
IoT data	❌	✅

- **SQL Databases**

MySQL
PostgreSQL
Oracle
SQL Server

- **NoSQL Databases**

MongoDB
Cassandra
Redis
DynamoDB
Neo4j

Projects Using SQL Databases

- **Example 1: Banking Systems**
- **Why SQL:**
  - Strong **ACID compliance** → critical for transactions
  - Structured data → accounts, customers, transactions
- **Project Example:**
  - Online banking application
  - Funds transfer, account balance updates, transaction history
- **Example 2: E-Commerce Websites**
- **Why SQL:**
  - Structured orders, inventory, users
  - Relationships between tables (customers ↔ orders ↔ products)
- **Project Example:**
  - Querying order history, generating invoices

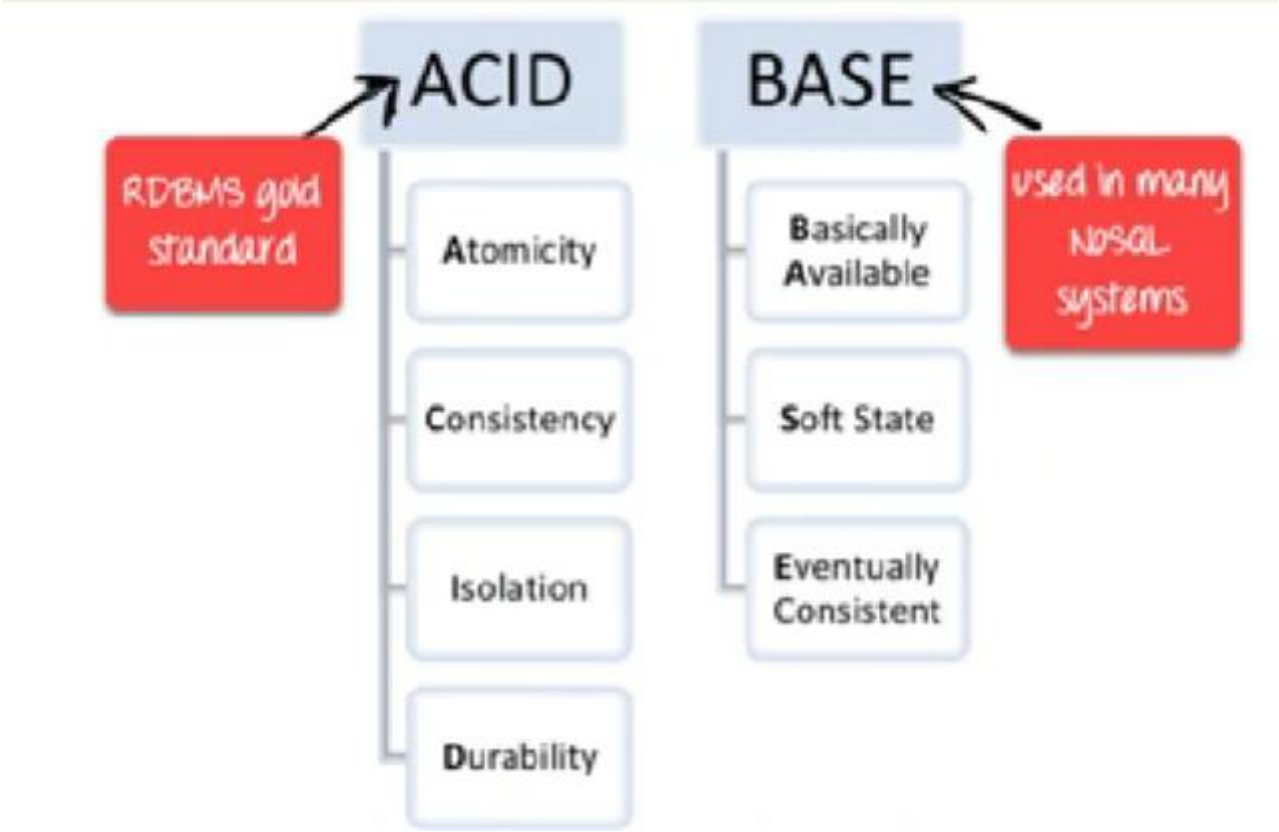
- **Summary for SQL Projects**
- Projects that need **transactional integrity, structured data, and complex relationships**

Projects Using NoSQL Databases

- **Example 1: Social Media Platforms**
- **Why NoSQL:**
  - Huge volume of user-generated data
  - Schema flexibility (posts, comments, likes, media files)
  - Horizontal scaling for millions of users
- **Project Example:**
  - Facebook posts/comments, Instagram feed
  - MongoDB storing posts with varying fields per user
- **Example 2: Real-Time Analytics / Big Data**
- **Why NoSQL:**
  - High write throughput
  - Distributed architecture → handles **terabytes of data**
- **Project Example:**
  - Netflix analytics (user watch history, recommendations)
  - Log streaming for clickstreams and metrics

- **Example 3: IoT & Sensor Data**
- **Why NoSQL:**
  - Time-series data → fast inserts
  - Flexible structure → different sensors produce different fields
- **Project Example:**
  - Smart home sensors, wearable devices
  - Storing temperature, motion, heartbeat readings
- **Example 4: E-Commerce Product Catalog**
- **Why NoSQL:**
  - Each product may have **different attributes**
  - No need for fixed schema
- **Project Example:**
  - Amazon product catalog
  - Products: Electronics vs Clothing vs Books → very different fields
- **Summary for NoSQL Projects**
- Projects that need **flexible schema, massive scale, distributed architecture, or high-speed writes**

ACID vs BASE


ACID
RDBMS gold standard
Atomicity
Consistency
Isolation
Durability
BASE
used in many NoSQL systems
Basically Available
Soft State
Eventually Consistent

ACID

Term	Meaning	Example
Atomicity	All operations in a transaction succeed **or none do**	Transferring money: Either $100 moves from A → B **completely** or not at all
Consistency	Database moves from one valid state to another	After transaction, total bank balance remains correct
Isolation	Transactions do not interfere with each other	Two users transferring money simultaneously won't corrupt data
Durability	Once committed, data persists even if system crashes	After transfer completes, data remains in DB even if server fails

BASE

- **Basically Available**

The system **always responds** to requests.

Even if some nodes fail, the system continues working.

Example:

Social media site still loads even if some servers are down.

- **Soft State**

The system state may **change over time**, even without new input.

Data may be temporarily inconsistent.

Example:

You like a post.

The like count may show 100 for you, but 98 for someone else for a few seconds.

- **Eventually Consistent**

All nodes will **eventually** “ in the last ” become consistent ” transactions in the middle is not so important”.

After some time, all users see the same data.

Example:

After a short delay, everyone sees the updated like count = 100.

BASE

Term	Meaning	Example
Basically Available	System is always available for reads/writes	Social media feed always shows posts, even if some nodes are down
Soft state	Data may change over time until fully consistent	Likes count on Instagram may be slightly off temporarily
Eventually consistent	All nodes will **eventually** reflect the same data	If you post a comment, it may take a few seconds to appear for everyone

Selecting DBMS
CAP Theorem

- Frist according to the data(structured, semi, non structured)

- Second according to CAP theorem:

In a distributed system, you can only guarantee **two out of three**:
Consistency, Availability, and Partition Tolerance.

You cannot guarantee all three at the same time.

- **Consistency (C)**

All nodes see the **same data at the same time**.

After writing data, every user reads the latest value.

Example:
If account balance becomes $500, every server shows $500 immediately.

- **Availability (A)**

The system always responds to requests.

Every request gets a response (success or failure).

Even if some servers fail, the system still works.

- **Partition Tolerance (P)**

The system continues working even if there is a **network failure between nodes**.

Nodes may lose communication, but system still operates.

Why Can't We Have All Three?

In distributed systems, network failures (partitions) are unavoidable.

So when partition happens, system must choose:

Keep **Consistency** and reject some requests Availability
OR

Keep **Availability** but allow temporary inconsistency Consistency

- **CP (Consistency + Partition Tolerance)**

Strong consistency
May reject requests during partition
Lower availability
Examples:
MongoDB (default mode)
HBase
Redis (cluster mode with strict config)
Used in: banking, financial systems

- **AP (Availability + Partition Tolerance)**

Always responds

May return outdated data

Eventual consistency

Examples:

Cassandra

DynamoDB

CouchDB

Used in: social media, large-scale web apps

- **CA (Consistency + Availability)**

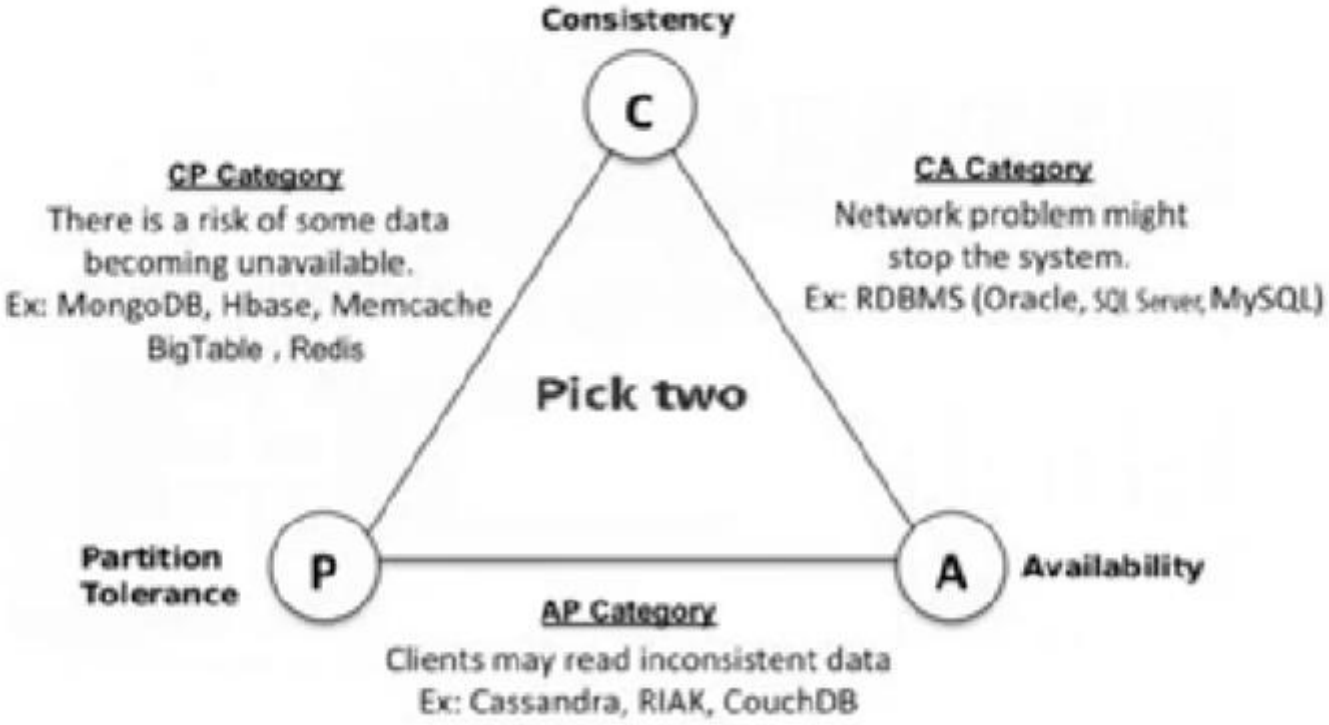
Works only when no partition exists

Not realistic for distributed systems

Examples:

Traditional SQL databases (single server)

MySQL (non-distributed setup)

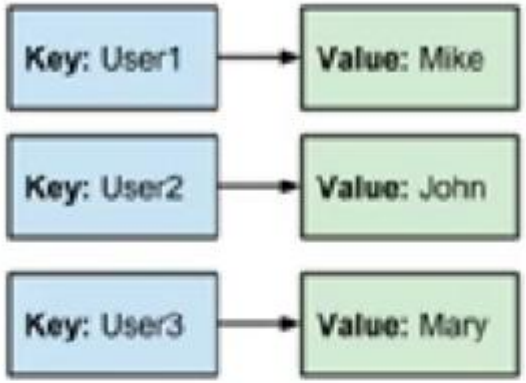

Consistency
C
CP Category
There is a risk of some data
becoming unavailable.
Ex: MongoDB, Hbase, Memcache
BigTable , Redis
CA Category
Network problem might
stop the system.
Ex: RDBMS (Oracle, SQL Server, MySQL)
Pick two
Partition
Tolerance
P
A
Availability
AP Category
Clients may read inconsistent data
Ex: Cassandra, RIAK, CouchDB

No SQL types

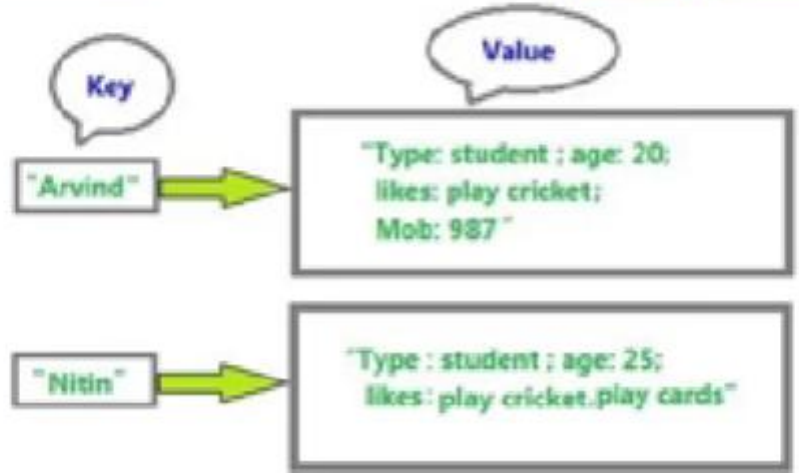
Type	Examples
Key-Value Store	redis, riak
Wide Column Store	HBASE, Cassandra
Document Store	mongoDB, CouchDB
Graph Store	Neo4j the graph database,

Key-value

- A **key-value store** is a non-relational database that stores data as simple key–value pairs, enabling fast and efficient data access.
- Stores each data item as a unique key and its corresponding value.
- Data is retrieved using the key.
- Values can be simple types or complex objects.
- Simplest form of NoSQL databases.
- Comparable to a relational table with only key and value columns.



Key	Value
K1	AAA,BBB,CCC
K2	AAA,BBB
K3	AAA,DDD
K4	AAA,2,01/01/2015
K5	3,ZZZ,5623



Features of the key-value store

- **Simplicity:** Data retrieval is extremely fast due to direct key access.
- **Scalability:** Designed for horizontal scaling and distributed storage.
- **Speed:** Ideal for caching and real-time applications.

Column-based

- A **column-oriented database** is a non-relational database that stores data by columns rather than rows, enabling efficient analytics and fast data retrieval.
- Stores data in columns instead of rows.
- Reads only required columns, reducing memory usage.
- Optimized for analytical queries.
- Provides faster data retrieval.
- Suitable for handling large volumes of data.

Row-Oriented vs Column-Oriented

Row-oriented: rows stored sequentially in a file

Key	Fname	Lname	State	Zip	Phone	Age	Sales
1	Bugs	Bunny	NY	11217	(123) 938-3235	34	100
2	Yosemite	Sam	CA	95389	(234) 375-6572	52	300
3	Buffy	Duck	NY	10013	(345) 227-3410	35	200
4	Elmer	Fudd	CA	94578	(456) 882-7123	43	10
5	Wilsh	Hazel	CA	11570	(567) 744-0991	67	250

Column-oriented: each column is stored in a separate file
Each column for a given row is at the same offset.

Key
1
2
3
4
5

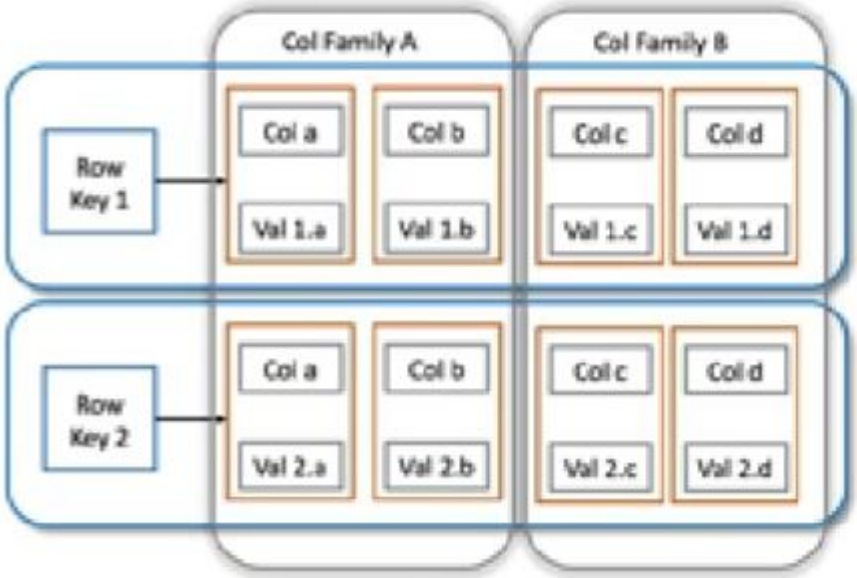
Fname
Bugs
Yosemite
Buffy
Elmer
Wilsh

Lname
Bunny
Sam
Duck
Fudd
Hazel

State
NY
CA
NY
CA
CA

Zip
11217
95389
10013
94578
11570

Phone
(123) 938-3235
(234) 375-6572
(345) 227-3410
(456) 882-7123
(567) 744-0991



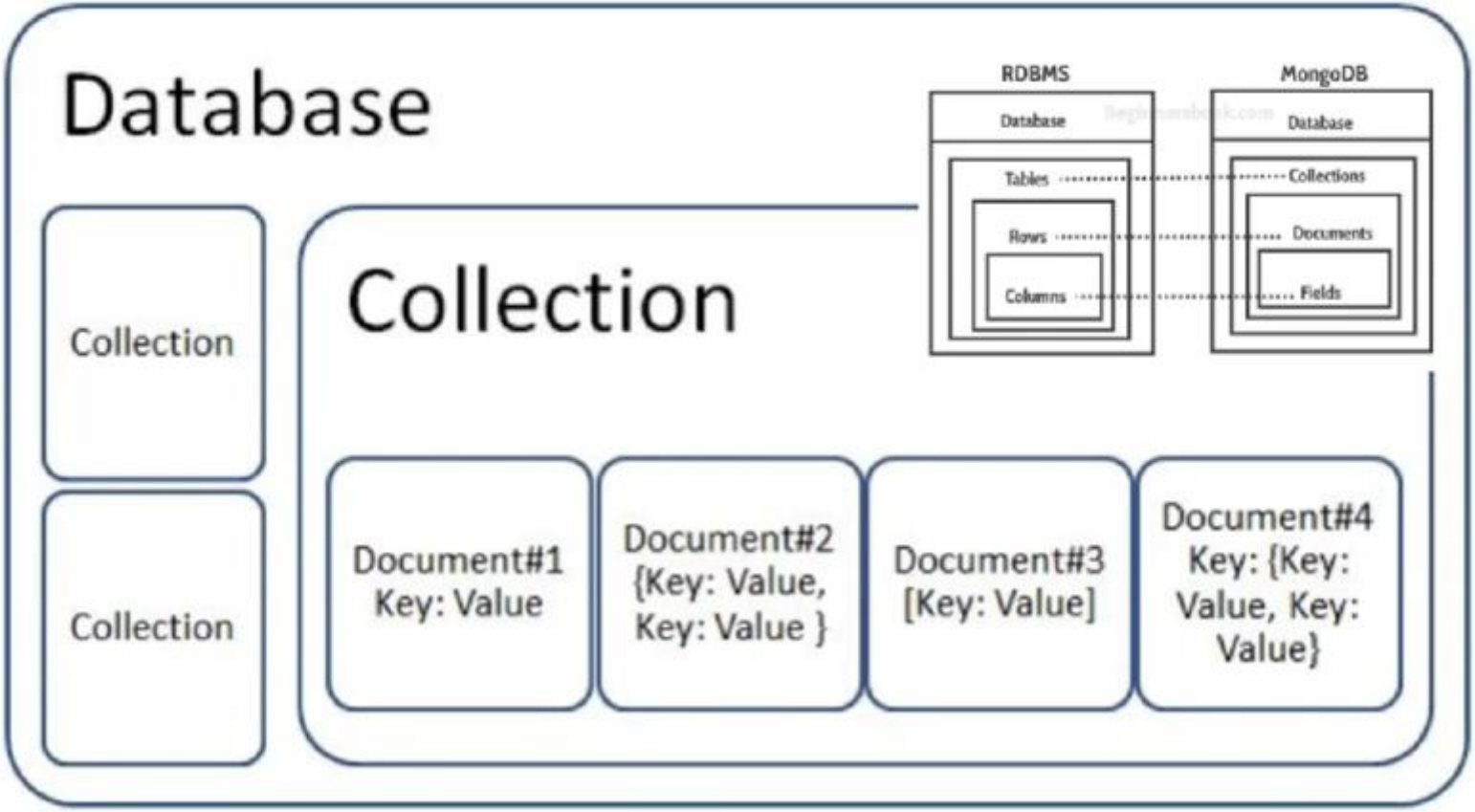
Features of Columnar Oriented Database

- **High Scalability:** Supports distributed data processing.
- **Compression:** Columnar storage enables efficient data compression.
- **Faster Query Performance:** Best for analytical queries.

Document- oriented

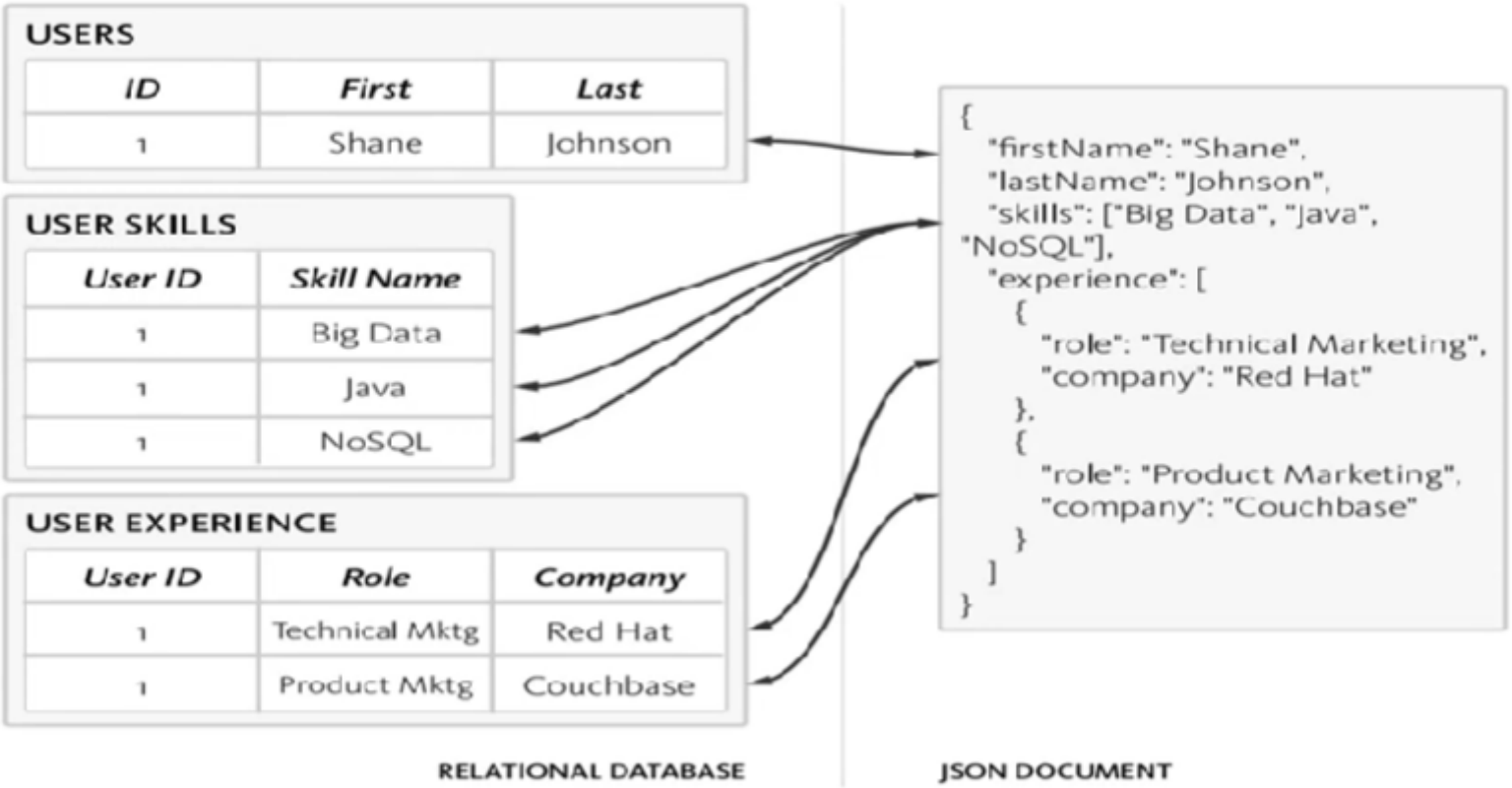
this course concern

- A **<u>document-based database</u>** is a non-relational database that stores data as documents, offering flexible schemas and efficient access aligned with application data models.
- Stores data in documents (JSON, BSON, or XML) instead of tables.
- Closely matches application data objects, reducing data translation.
- Supports indexing for faster access to specific document fields.
- Uses collections to group related documents with similar content.
- Allows flexible schemas, so documents in a collection need not be identical.


Database
Collection
Collection
Collection
Document#1
Key: Value
Document#2
{Key: Value,
Key: Value }
Document#3
[Key: Value]
Document#4
Key: {Key:
Value, Key:
Value}
RDBMS
Database
Tables
Rows
Columns
MongoDB
Database
Collections
Documents
Fields

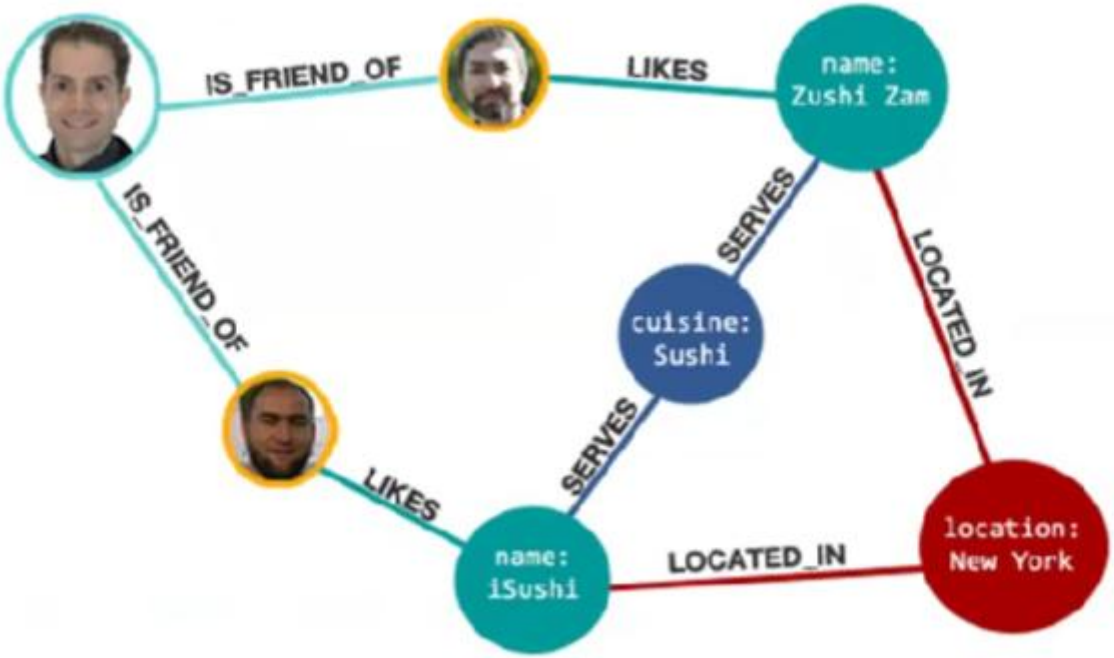
Features of document database

- **Flexible schema:** Documents in the database has a flexible schema. It means the documents in the database need not be the same schema.
- **Faster creation and maintenance:** the creation of documents is easy and minimal maintenance is required once we create the document.
- **No foreign keys:** Documents are independent, so relationships are not enforced and foreign keys are not required.
- **Open formats**: To build a document we use XML, JSON, and others.


USERS
ID
First
Last
1
Shane
Johnson
USER SKILLS
User ID
Skill Name
1
Big Data
1
Java
1
NoSQL
USER EXPERIENCE
User ID
Role
Company
1
Technical Mktg
Red Hat
1
Product Mktg
Couchbase
{
"firstName": "Shane",
"lastName": "Johnson",
"skills": ["Big Data", "Java",
"NoSQL"],
"experience": [
{
"role": "Technical Marketing",
"company": "Red Hat"
},
{
"role": "Product Marketing",
"company": "Couchbase"
}
]
}
RELATIONAL DATABASE
JSON DOCUMENT

Graph-based

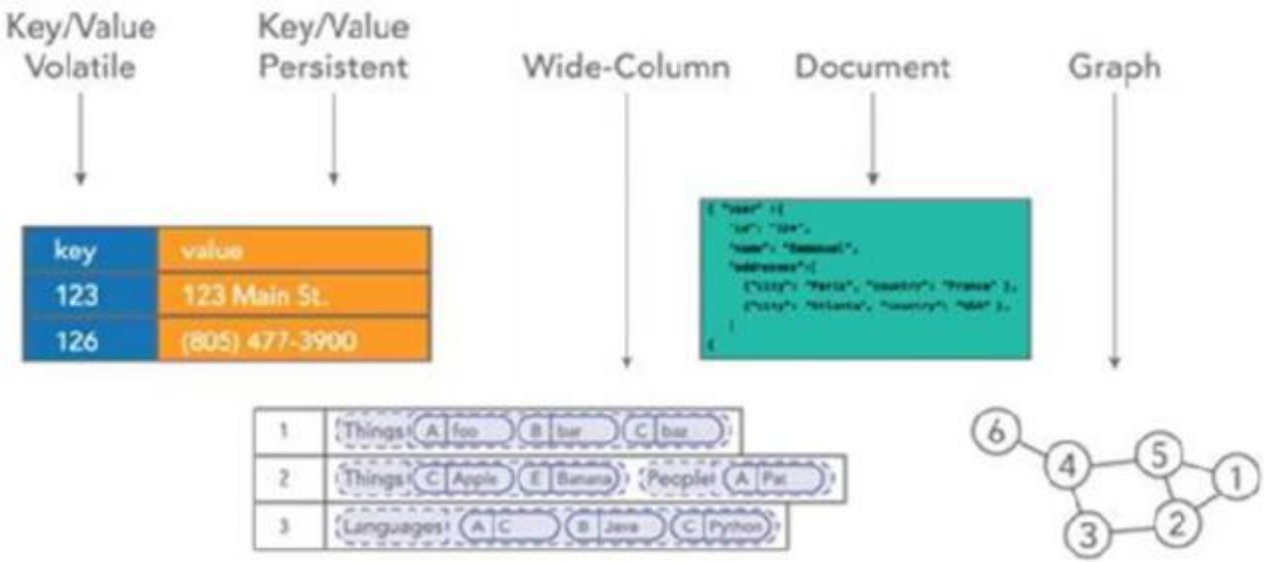
- **Graph-based** databases store data as nodes and relationships, making them ideal for managing and querying complex, interconnected data.
- Data is represented as nodes (objects) and edges (connections).
- Fast graph traversal algorithms help retrieve relationships quickly.
- Used in scenarios where relationships are as important as the data itself.


IS_FRIEND_OF
LIKES
name:
Zushi Zam
IS_FRIEND_OF
SERVES
LOCATED_IN
cuisine:
Sushi
SERVES
LIKES
name:
iSushi
LOCATED_IN
location:
New York

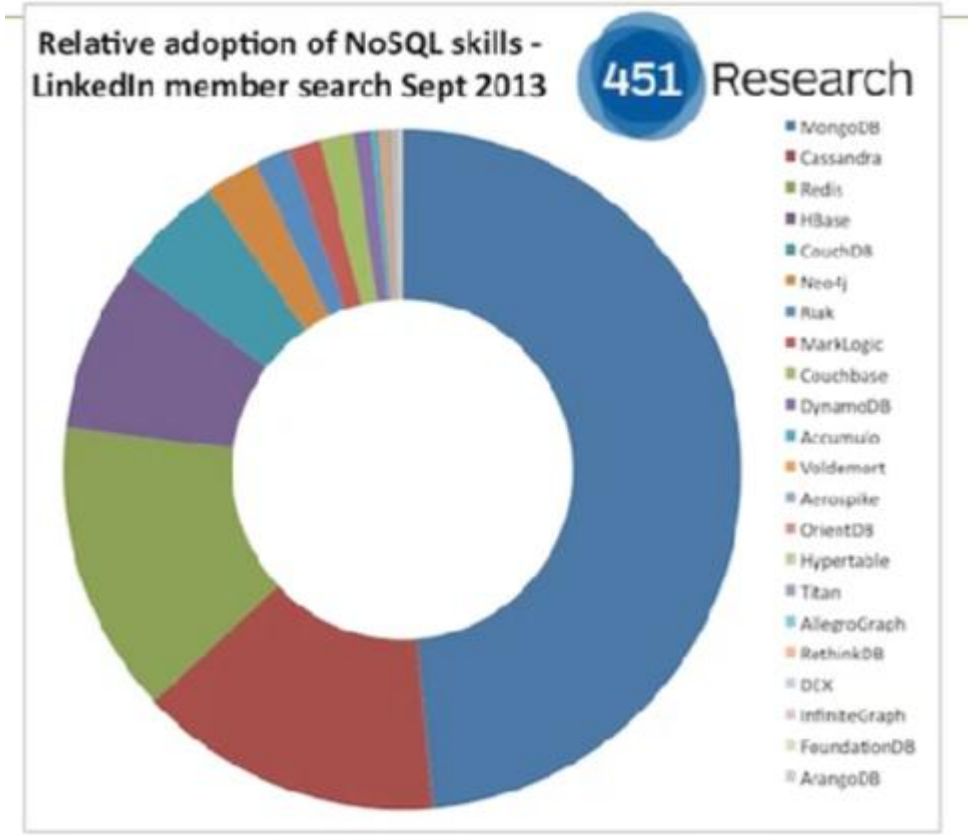
Features of Graph Database

- **Relationship-Centric Storage:** Perfect for social networks, fraud detection, recommendation engines.
- **Real-Time Query Processing:** Queries return results almost instantly.
- **Schema Flexibility:** Easily adapts to evolving relationship structures

- Summary


Key/Value
Volatile
Key/Value
Persistent
Wide-Column
Document
Graph
key
value
123
123 Main St.
126
(805) 477-3900
1
Things
A
foo
B
bar
C
baz
2
Things
C
Apple
E
Banana
People
A
Pat
3
Languages
A
C
B
Java
C
Python
6
4
5
1
3
2

- Top NoSQL used



MongoDB
Document-oriented DB

MongoDB

- **MongoDB** is

Source-available,

Cross-platform” works on any platform windows/linux…”,

Document-oriented database program.

Classified as a NoSQL database product, MongoDB uses JSON-like documents (called BSON) with schema free.

BSON (Binary JSON)

- It is a binary-encoded format used to **store** and **transfer** data, mainly with MongoDB.
- BSON is like JSON, but:

Stored in **binary format**

Faster for machines to read/write

Supports **more data types** than JSON

- Supports additional types like:

Date, Binary data, ObjectId, 64-bit integers, Decimal128

Example:

- Json

```json
{
  "name": "Youssef",
  "age": 22
}
```

- BSON

Stored in binary form (not readable as text)

- **Advantages**

Faster processing
More data types
Better for large-scale databases

- **Disadvantages**

Not human-readable

Create DB in Mongo
Using cmd ”command line”

In MongoDB, a database is created **automatically** when you first store data in it.

```
use myDatabase
```

Example:

```
use universityDB
```

If the database does not exist, MongoDB will prepare to create it.

Create a Collection (Insert Data)

- MongoDB only actually creates the database when you insert data:

```
db.students.insertOne({
  name: "Youssef",
  age: 22,
  major: "Computer Science"
})
```

- Now the database universityDB is created.

- Show Databases `show dbs`
- Check Current Database `db`

Note: MongoDB does **NOT** create empty databases.

A database exists only after:

A collection is created

At least one document is inserted

Create Database in MongoDB Compass (GUI)

- **Step 1: Open MongoDB Compass**

Launch MongoDB Compass.
Connect to your MongoDB server (usually:

```
mongodb://localhost:27017
```

- **Step 2: Click "Create Database"**

On the main screen, click the **Create Database** button.

- **Step 3: Enter Details**

You must enter:
Database Name → example: universityDB
Collection Name → example: students
MongoDB requires at least one collection to create a database.

- **Step 4: Click Create Database**

Now your database is created and will appear in the left panel.